ANN (Artificial Neural Networks)

Structured Notes

Perceptron Abstraction, Shape Tracking, Non-Linearity,
Activation Functions, and `nn.Module` Implementation

Contents

1 From Biological Neuron to Artificial Neuron **2**

2 Single Neuron in Tensor Form **2**
2.1 Shape Setup 2

3 Full Layer with Multiple Neurons **3**
3.1 Why Weight Matrix Becomes Two-Dimensional 3

4 Two-Layer ANN: Dimensional Derivation **3**

5 Why Non-Linearity Is Necessary **4**
5.1 Linear Layer Stacking Collapse 4

6 Activation Functions and Gradient Behavior **5**
6.1 Sigmoid 5
6.2 Tanh 5
6.3 ReLU 5
6.4 Comparison Snapshot 5

7 Hidden Layer Interpretation **5**

8 Implementation with `nn.Module` **5**
8.1 Key Storage Convention in `nn.Linear` 5

9 Shape Debugging Checklist **6**

1 From Biological Neuron to Artificial Neuron

An artificial neuron is a mathematical abstraction of biological neural behavior. The abstraction keeps three core operations:

1. linear combination of inputs,
2. bias shift,
3. non-linear activation.

For one input vector $x \in \mathbb{R}^D$, one neuron uses weights $w \in \mathbb{R}^D$ and bias $b \in \mathbb{R}$.

Single-neuron forward computation:

$$z = w^T x + b$$

$$a = \phi(z)$$

Interpretation:

- $w^T x$: projection / linear transformation,
- $+b$: shift,
- $\phi(\cdot)$: non-linear mapping.

2 Single Neuron in Tensor Form

2.1 Shape Setup

For a mini-batch with N samples and D features:

$$X \in \mathbb{R}^{N \times D}, \quad w \in \mathbb{R}^D, \quad b \in \mathbb{R}$$

Then:

$$Z = Xw + b \in \mathbb{R}^N$$

After activation:

$$A = \phi(Z) \in \mathbb{R}^N$$

Activation changes values, not tensor shape.

Listing 1: Single-neuron forward pass with shape check

```python
import torch

N, D = 4, 3
X = torch.randn(N, D)       # [4, 3]
w = torch.randn(D)          # [3]
b = torch.randn(1)          # [1]

Z = X @ w + b               # [4]
A = torch.relu(Z)           # [4]

print("X shape:", X.shape)
print("w shape:", w.shape)
print("b shape:", b.shape)
print("Z shape:", Z.shape)
print("A shape:", A.shape)
```

For one neuron, w is tied to feature dimension (D), not sample count (N). Using N by mistake causes shape mismatch.

3 Full Layer with Multiple Neurons

3.1 Why Weight Matrix Becomes Two-Dimensional

When a layer contains M neurons, each neuron has its own weight vector of length D and one bias value.

$$X \in \mathbb{R}^{N\times D}, \quad W \in \mathbb{R}^{M\times D}, \quad b \in \mathbb{R}^{M}$$

Layer linear transformation:

$$Z = XW^T + b$$

Shape flow:

$$(N \times D)(D \times M) + (M) = N \times M$$

Final activation:

$$A = \phi(Z) \in \mathbb{R}^{N\times M}$$

After linear transformation, rows correspond to data samples and columns correspond to neuron outputs (transformed features).

Listing 2: Single layer with multiple neurons

```python
import torch

N, D, M = 5, 3, 4
X = torch.randn(N, D)           # [5, 3]
W = torch.randn(M, D)           # [4, 3]
b = torch.randn(M)              # [4]

Z = X @ W.T + b                 # [5, 4]
A = torch.relu(Z)               # [5, 4]

print("X:", X.shape, "W:", W.shape, "b:", b.shape)
print("Z:", Z.shape, "A:", A.shape)
```

4 Two-Layer ANN: Dimensional Derivation

Define:

$$D = \text{input dimension}, \quad H = \text{hidden layer size}, \quad O = \text{output dimension}$$

Parameters:

$$W_1 \in \mathbb{R}^{H\times D}, \quad b_1 \in \mathbb{R}^{H}$$
$$W_2 \in \mathbb{R}^{O\times H}, \quad b_2 \in \mathbb{R}^{O}$$

Forward pass:

$$Z_1 = XW_1^T + b_1 \in \mathbb{R}^{N\times H}$$
$$A_1 = \phi(Z_1) \in \mathbb{R}^{N\times H}$$
$$Z_2 = A_1W_2^T + b_2 \in \mathbb{R}^{N\times O}$$

Output dimension O depends on task design:

- $O = 1$ for single-value regression,
- $O = C$ for C-class classification logits.

Listing 3: Two-layer ANN with explicit shape tracking

```python
import torch

N, D, H, O = 6, 2, 4, 1
X  = torch.randn(N, D)
W1 = torch.randn(H, D)
b1 = torch.randn(H)
W2 = torch.randn(O, H)
b2 = torch.randn(O)

Z1 = X @ W1.T + b1              # [6, 4]
A1 = torch.relu(Z1)             # [6, 4]
Z2 = A1 @ W2.T + b2             # [6, 1]

print("Z1:", Z1.shape, "A1:", A1.shape, "Z2:", Z2.shape)
```

5 Why Non-Linearity Is Necessary

5.1 Linear Layer Stacking Collapse

Without activation:

$$y = W_2(W_1x + b_1) + b_2$$

$$y = (W_2W_1)x + (W_2b_1 + b_2)$$

This is still a single affine transformation.

Depth without activation does not add expressive power. Multiple linear layers collapse into one linear layer.

Listing 4: Numerical verification of linear collapse

```python
import torch

torch.manual_seed(42)
D, H, O = 3, 5, 2
W1 = torch.randn(H, D)
b1 = torch.randn(H)
W2 = torch.randn(O, H)
b2 = torch.randn(O)

W_prime = W2 @ W1
b_prime = W2 @ b1 + b2

X = torch.randn(7, D)
out_two_layer = (X @ W1.T + b1) @ W2.T + b2
out_single    = X @ W_prime.T + b_prime

print((out_two_layer - out_single).abs().max())
# Very close to 0 (floating-point precision)
```

6 Activation Functions and Gradient Behavior

6.1 Sigmoid

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma(z) \in (0,1)$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

For large $|z|$, derivative becomes very small, creating vanishing gradients.

6.2 Tanh

$$\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}, \quad \tanh(z) \in (-1,1)$$

$$\frac{d}{dz}\tanh(z) = 1-\tanh^2(z)$$

Tanh is zero-centered but still saturates for large magnitude inputs.

6.3 ReLU

$$\mathrm{ReLU}(z) = \max(0,z)$$

$$\frac{d}{dz}\mathrm{ReLU}(z) = \begin{cases} 1, & z>0 \\ 0, & z<0 \end{cases}$$

ReLU avoids saturation on the positive side but can produce inactive neurons when many activations stay negative.

Common variants used to reduce dead-neuron behavior include Leaky ReLU, SELU, and GELU.

6.4 Comparison Snapshot

Activation	Range	Zero-Centered	Saturation Risk
Sigmoid	$(0,1)$	No	High (both tails)
Tanh	$(-1,1)$	Yes	High (both tails)
ReLU	$[0,\infty)$	No	Low on positive side

7 Hidden Layer Interpretation

Each neuron defines a linear hyperplane in feature space. The layer projects data into new coordinates; activation then bends this representation into a non-linear feature map.

The hidden layer can be viewed as feature construction: original features are transformed into neuron responses. This projection intuition is related to hyperplane-based methods, while remaining distinct from PCA's variance-maximizing dimensionality reduction objective.

8 Implementation with `nn.Module`

8.1 Key Storage Convention in `nn.Linear`

`nn.Linear(in_features, out_features)` stores weight as:

$$W \in \mathbb{R}^{\text{out_features} \times \text{in_features}}$$

Forward internally computes:

$$XW^T + b$$

No manual transpose is required in user code.

Listing 5: Two-layer ANN using nn.Module and intermediate outputs

```python
import torch
import torch.nn as nn

class TwoLayerANN(nn.Module):
    def __init__(self, d_in, h, d_out):
        super().__init__()
        self.fc1 = nn.Linear(d_in, h)
        self.fc2 = nn.Linear(h, d_out)

    def forward(self, x):
        z1 = self.fc1(x)
        a1 = torch.relu(z1)
        z2 = self.fc2(a1)
        return z1, a1, z2

X = torch.randn(5, 2)
model = TwoLayerANN(d_in=2, h=4, d_out=1)
z1, a1, y_hat = model(X)

print("X:", X.shape)
print("z1:", z1.shape)
print("a1:", a1.shape)
print("y_hat:", y_hat.shape)
print("fc1 weight:", model.fc1.weight.shape)  # [4, 2]
print("fc2 weight:", model.fc2.weight.shape)  # [1, 4]
```

9 Shape Debugging Checklist

When building ANN code, use this checklist:

1. Confirm input shape is $[N, D]$.
2. For each linear layer, confirm weight shape is $[\text{out}, \text{in}]$.
3. Validate matrix multiplication rule: inner dimensions must match.
4. Confirm bias length equals number of neurons in that layer.
5. Verify activation preserves tensor shape.
6. For stacked linear layers, confirm hidden/output dimensions are propagated correctly.
7. Inspect intermediate tensors (Z_1, A_1, Z_2) when debugging.

Frequent concept checks:

- If $X \in \mathbb{R}^{N \times D}$ and output after first layer is $N \times H$, then W_1 is $H \times D$.
- Removing activation from multi-layer ANN turns it into an affine map.

- ReLU does not change shape; it changes values.